

Security Audit

BTX Markets (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	12
Dependencies	12
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Conclusion	75
Our Methodology	77
Disclaimers	79
About Hashlock	80

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The BlackTie Deal team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BTX Markets is the secondary marketplace of the Black Tie Asset Hub, designed as a non-custodial, blockchain-based trading environment for tokenised real-world assets. Built on the Ethereum network, BTX Markets enables peer-to-peer buying, selling, and trading of digital asset tokens with a focus on transparency, security, and efficiency. The platform integrates robust smart contract protocols to facilitate trustless transactions while maintaining compliance and auditability standards required for institutional-grade operations.

Project Name: BlackTie Deal

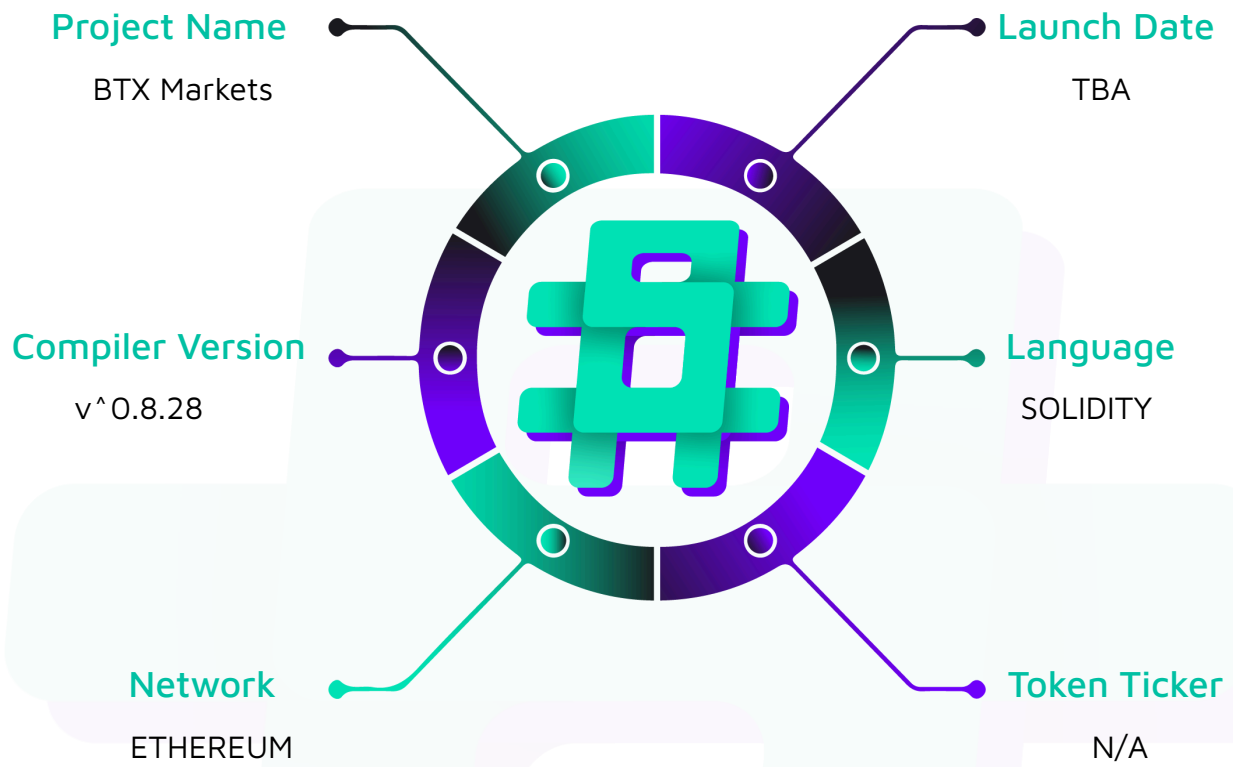
Project Type: dApp

Compiler Version: ^0.8.28

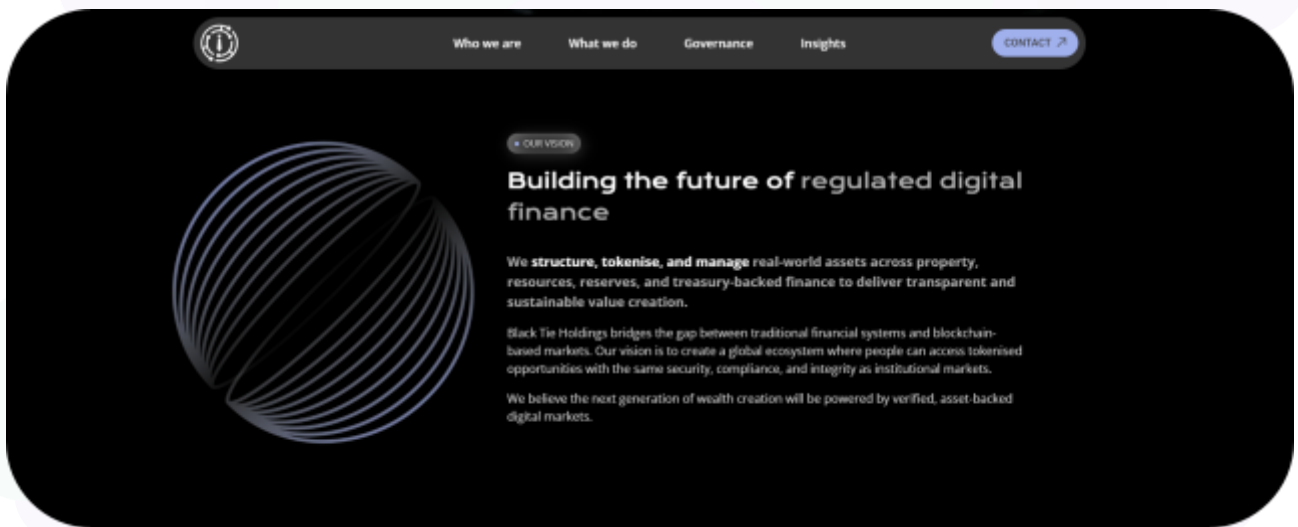
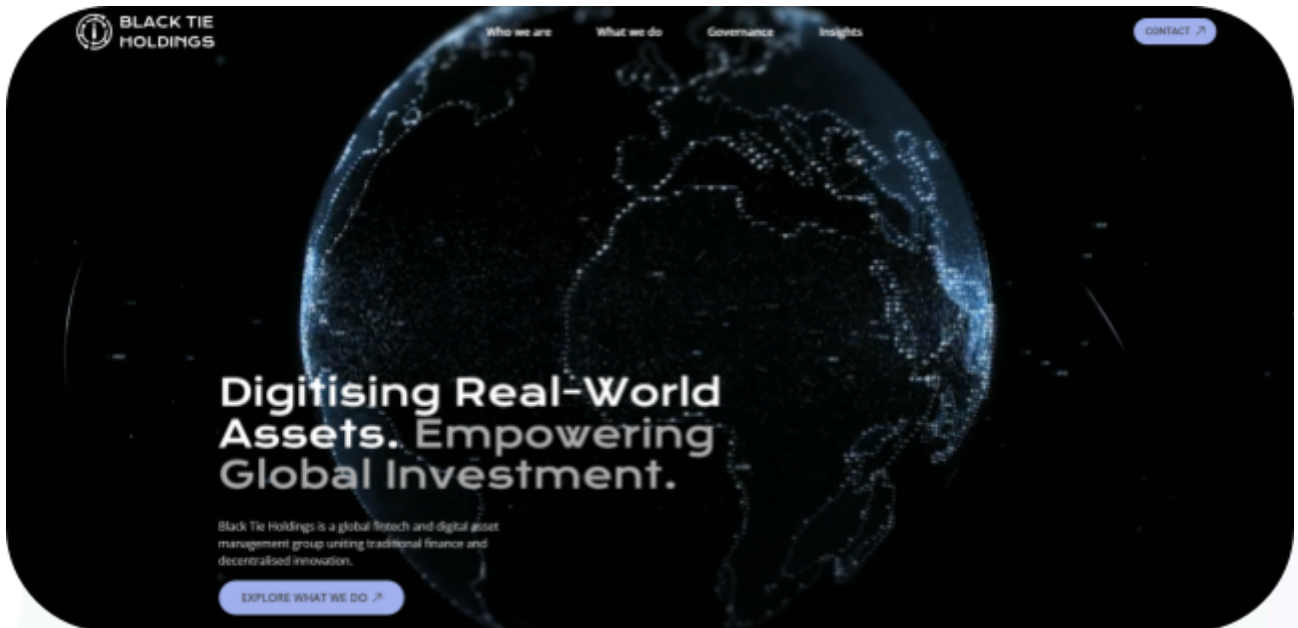
Website: <https://blacktie.digital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the BTX Markets project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	BTX Markets Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	April, 2026
Contract 1	P2P.sol
Contract 2	P2PLibrary.sol
Contract 3	ProxyContract.sol
Audited GitHub Commit Hash	7262505fd4c7d6f5394d31be6931b4d161a86157
Fix Review GitHub Commit Hash	09e02645fc28a72ead0d3339eebb07767f62d2ed

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

6 Medium severity vulnerabilities

11 Low severity vulnerabilities

10 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
P2P.sol - Allows the owner to: - Set buy/sell fees and price markup - Set authority manager, factory, gold/silver price aggregators - Allows buyers to: : - Create buy offers (escrow USDC/USDT upfront, specify token, price, amount) - Cancel open or partially filled buy offers and receive remaining payment back - Allows sellers to: - Create sell offers (escrow Gold/Silver/ERC3643 tokens, specify price, payment token) - Cancel open or partially filled sell offers and receive remaining tokens back - Allows sellers to: - Fill a buy offer partially or fully by supplying tokens and receiving USDC/USDT - Fill a sell offer partially or fully by supplying USDC/USDT and receiving tokens - Query current price for Gold or Silver from Chainlink oracles - Query fee and payment calculations via public view functions	Contract achieves this functionality.

ProxyContract.sol - Allows the proxy owner to: - Deploy an upgradeable proxy pointing to an implementation contract - Upgrade the implementation via <code>upgradeToAndCall()</code> (emits <code>Upgraded</code> event) - Swap the implementation silently via <code>setImplementation()</code> (no event) - Transfer proxy ownership to a new address (single-step) - Renounce proxy ownership - Delegates all non-admin calls to the current implementation via <code>fallback()</code> Accepts plain ETH via <code>receive()</code> (no withdrawal mechanism)	Contract achieves this functionality.
P2PLibrary.sol - Provides shared data types for the P2P protocol: - Enums: <code>PropertyType</code> (<code>ERC20</code>, <code>ERC3643</code>), <code>OrderStatus</code> (<code>PENDING</code>, <code>PARTIALLY_FILLED</code>, <code>FILLED</code>, <code>CANCELLED</code>), <code>BuyType</code> (<code>USDT</code>, <code>USDC</code>), <code>AssetType</code> (<code>Gold</code>, <code>Silver</code>) - Struct <code>BuyOffer</code>: stores price, token amounts, payment amounts, fee accumulator, payment token, property type, order status, maker address, token address - Struct <code>SellOffer</code>: stores price, token amounts, fee accumulator, payment token, property type, order status, maker address, token address	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Struct TokenInfo: stores factory-registered token metadata (agent, address, decimals, supply limits, asset type)- Struct PropertyData: stores ERC3643 property metadata (price, decimals, supply, limits, agent/owner/token addresses)- Contains no logic — pure type definitions consumed by P2P.sol | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the BlackTie Deal project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the BlackTie Deal project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Zero-Payment Sell Offer via Precision Truncation — Free Token Extraction

Description

`calculateAmountBuyerPays` contains no guard requiring `buyAmount > 0` after integer division. For tokens with 18 decimals, small `_tokenAmount` values truncate to zero USDC payment. The buyer's token transfer still executes, allowing an attacker to drain a sell offer for free.

Vulnerability Details

```
// P2P.sol:580-585
buyAmount =
    (_tokenAmount * _offer.price * (10 ** buyTokenDecimals)) /
    (10 ** tokenDecimals * 10 ** 6);

fee = (buyAmount * sellFee) / 1e10;
buyAmount += fee;
```

For an 18-decimal token at price \$2,000 (`_offer.price = 2e9` in D6 format):

```
buyAmount = (_tokenAmount * 2e9 * 1e6) / (1e18 * 1e6)
            = _tokenAmount * 2e9 / 1e18
            = _tokenAmount * 2e-9
```

Any `_tokenAmount < 5e8` (500 million smallest units) produces `buyAmount = 0`. No `require(buyAmount > 0)` check exists.

With `buyAmount = 0`, `_totalFee = 0`, `_sellerReceive = 0`:

```
// P2P.sol:527-538
```

```

_buyToken.safeTransferFrom(msg.sender, authorityManager.owner(), 0); // OK - no USDC
paid
_buyToken.safeTransferFrom(msg.sender, _offer.maker, 0); // OK - no USDC
paid
IERC20Metadata(_offer.tokenAddress).safeTransfer(msg.sender, _tokenAmount); // tokens
transferred!

```

Proof of Concept

```

// Attacker calls acceptSellOffer repeatedly with small _tokenAmount
// On an L2 with cheap gas (e.g., Arbitrum, ~$0.01/tx):

for (uint i = 0; i < 499_999_999; i++) {
    p2p.acceptSellOffer(sellOfferId, 1); // pays 0 USDC, receives 1 token unit
}

// Total cost: gas only. Total received: 499,999,999 token units.

```

Impact

Complete theft of any sell offer containing tokens with ≥ 18 decimals. ERC3643 property tokens (via `property.token`) may use 18 decimal precision. ERC20 sell orders additionally accept arbitrary tokens (see L-04), enabling this attack against any 18-decimal token placed in escrow. On L2 chains with gas $< \$0.01/\text{tx}$, the attack is economically viable even for moderate-value orders.

Recommendation

Add a zero-payment guard immediately after the formula:

```

buyAmount =
    (_tokenAmount * _offer.price * (10 ** buyTokenDecimals)) /
    (10 ** tokenDecimals * 10 ** 6);

require(buyAmount > 0, "Payment rounds to zero - increase fill amount");

fee = (buyAmount * sellFee) / 1e10;
buyAmount += fee;

```

Additionally, enforce a minimum fill amount at the offer level to bound the worst-case precision loss.

Status

Resolved



[H-02] Token Address Decimal Mismatch Between Offer Creation and Fill

Description

`createBuyOffer` stores the user-supplied `_tokenAddress` in the offer struct, but `calculateForBuyOffer` reads token decimals from the factory's canonical address (`info.tokenAddress`). At fill time, `calculateAmountSellerGets` reads decimals from the stored (user-supplied) address. If the factory's canonical address and the user-supplied address have different decimal counts, the payment amount can be miscalculated by a factor of up to 10^{12} .

Vulnerability Details

****At creation**** — decimals read from factory canonical address:

```
// P2P.sol:331-340 (calculateForBuyOffer)
P2PLibrary.TokenInfo memory info = factory.getTokenInfo(_tokenAddress);
tokenAddress = info.tokenAddress; // factory canonical address
// ...
IERC20Metadata token = IERC20Metadata(tokenAddress); // ← uses factory address
uint256 tokenDecimals = token.decimals();           // e.g., 18 from factory
buyAmount = (_tokenAmount * _price * 1e6) / (10**tokenDecimals * 1e6);
```

****Stored in struct**** — user-supplied address:

```
// P2P.sol:169
_offer.tokenAddress = _tokenAddress; // ← user-supplied, not factory canonical
```

****At fill**** — decimals read from stored (user-supplied) address:

```
// P2P.sol:477-484 (calculateAmountSellerGets)
IERC20Metadata _token = IERC20Metadata(_offer.tokenAddress); // ← stored (user)
address
uint256 tokenDecimals = _token.decimals();                     // e.g., 6 from user
address
amountSellerGets = (_tokenAmount * _offer.price * 1e6) / (10**tokenDecimals * 1e6);
```

****Impact on payment amounts**** (factory canonical = 18 decimals, user-supplied = 6 decimals):

```

Creation pre-funds:  buyAmount = (100 * $3000 * 1e6) / (1e18 * 1e6) = 0 (rounds to 0)
Fill calculates:      amountSellerGets = (100 * $3000 * 1e6) / (1e6 * 1e6) = $300

Seller attempts fill → remainingPaymentAmount -= ($300 + fee) → UNDERFLOW REVERT
→ Order permanently unfillable

```

Proof of Concept

1. Factory registers GOLD token at canonical address `0xCANON` (18 decimals).
2. User creates buy order using `_tokenAddress = 0xPROXY` (same token, proxy/wrapper at 6 decimals).
3. Factory's `getTokenInfo(0xPROXY)` returns `info.tokenAddress = 0xCANON` (18 decimals) — addresses differ.
4. Order creation succeeds, pre-funding calculated using 18-decimal math.
5. Any fill attempt uses 6-decimal math → `amountSellerGets` is $10^{12} \times$ larger → underflow → revert.
6. Order is permanently stuck — neither fillable nor cancellable (cancel refunds USDC but tokens were never deposited).

Impact

Orders created with a token address that differs from the factory canonical are permanently unfillable. Buyer's pre-funded USDC is locked until `cancelBuyOffer` is called. If `cancelBuyOffer` has a separate bug or the user is unaware, funds are stuck.

Recommendation

Store the factory-canonical address in the offer struct, not the user-supplied address:

```

// In createBuyOffer, after calculateForBuyOffer:
P2PLibrary.TokenInfo memory info = factory.getTokenInfo(_tokenAddress);
_offer.tokenAddress = _propertyType == P2PLibrary.PropertyType.ERC3643
    ? property.token
    : info.tokenAddress; // ← use factory canonical, not _tokenAddress

```

Status

Resolved

Medium

[M-01] Stale Chainlink Oracle — No Freshness Validation

Description

`getAssetPrice()` calls `latestRoundData()` on the Chainlink aggregator but discards all return values except `price`. There is no check for round completeness, data age, or heartbeat expiry. All order price validation relies on a potentially stale oracle value.

Vulnerability Details

```
// P2P.sol:373
(, int256 price, , , ) = priceAggregator.latestRoundData();
require(price > 0, "Invalid oracle price");
```

Missing standard Chainlink freshness validations:

- `updatedAt` — timestamp of last oracle update; no heartbeat check
- `answeredInRound >= roundId` — ensures the round is complete, not in-progress

`getAssetPrice` is called inside `calculateForBuyOffer` (L334) and directly in `createSellOffer` (L206). The stored `offer.price` is derived from this stale value and used for all subsequent fill calculations with no re-validation.

Proof of Concept

1. Chainlink XAU/USD feed lags during high volatility. Real gold price: \$2,600. Oracle still reports: \$3,000.
2. Malicious seller creates sell order at \$3,000 — passes `_validatePrice` against stale oracle.
3. Unsuspecting buyer calls `acceptSellOffer`, pays \$3,300/token (with 10% fee).
4. Buyer overpays by \$700/token. Attacker profits from staleness event.

Impact

Economic harm to buyers and sellers during Chainlink staleness events (documented on mainnet during volatility spikes). Impact bounded by `priceMarkup` setting (default 10%), but real price divergence can exceed this.

Recommendation

```
(uint80 roundId, int256 price, , uint256 updatedAt, uint80 answeredInRound)
    = priceAggregator.latestRoundData();

require(price > 0, "Invalid oracle price");
require(answeredInRound >= roundId, "Stale price: incomplete round");
require(block.timestamp - updatedAt <= 3600, "Stale price: heartbeat exceeded");
// 3600s matches Chainlink XAU/USD and XAG/USD heartbeat
```

Status

Resolved

[M-02] Fee Rate Not Snapshotted at Offer Creation — DOS or Surplus Extraction

Description

`buyFee` and `sellFee` are global variables read at fill time, not at order creation. `createBuyOffer` pre-funds based on the fee at creation time; `acceptBuyOffer` deducts based on the fee at fill time. A fee change between these two moments either causes all open buy orders to revert (underflow DOS) or silently extracts the buyer's pre-funded fee deposit to the owner.

Vulnerability Details

****At creation**** — fee snapshotted into `paymentAmount`:

```
// P2P.sol:141-156
(uint256 paymentAmount, uint256 fee) = calculateForBuyOffer(...);
// calculateForBuyOffer:350: fee = (buyAmount * buyFee) / 1e10 ← buyFee at creation
_offer.paymentAmount = paymentAmount + fee;
_offer.remainingPaymentAmount = paymentAmount + fee;
```

****At fill**** — fee read from CURRENT global state:

```
// P2P.sol:483
fee = (amountSellerGets * buyFee) / 1e10; // ← buyFee at fill time - may differ
```

****Scenario A — fee raised (permanent DOS):****

```
createBuyOffer at buyFee=10%: remainingPaymentAmount = $11,000

Owner raises buyFee to 50%.

acceptBuyOffer:
  _totalDeduction = $10,000 * 1.5 + fee = $16,000
  L415: remainingPaymentAmount = $11,000 - $16,000 → UNDERFLOW → REVERT
```

Every open buy order becomes permanently unfillable until each maker individually cancels.

****Scenario B — fee lowered (silent extraction):****

```
createBuyOffer at buyFee=10%: remainingPaymentAmount = $11,000 (base $10,000 + $1,000 fee)
```

Owner lowers buyFee to 0%.

```
acceptBuyOffer (full fill):
```

```
  _totalDeduction = $10,000 (no fee)
```

```
  remainingPaymentAmount = $11,000 - $10,000 = $1,000
```

```
filledTokens == totalTokens → L439: safeTransfer(owner, $1,000) ← buyer loses $1,000
```

The same asymmetry exists for sell orders: a post-creation fee increase silently reduces the seller's payout below the listed price with no revert and no notification.

Proof of Concept

1. Alice creates a buy order for 200 GOLD at \$50 (buyFee = 10%), pre-funds \$11,000 USDC.
2. Owner calls `setBuyFee(0)`.
3. Bob fills the entire order.
4. `_totalDeduction = $10,000`. `remainingPaymentAmount = $1,000`.
5. `filledTokens == totalTokens → safeTransfer(owner, $1,000)`.
6. Alice's \$1,000 fee deposit is sent to the owner. Alice receives no refund.

Impact

Fee increase freezes all open buy orders simultaneously — a single owner transaction causes protocol-wide DOS. Fee decrease silently extracts every buyer's pre-funded fee to the owner. The impact scales with the number and size of open orders.

Recommendation

Snapshot `buyFee/sellFee` into the offer struct at creation time and use the stored value at fill time:

```
struct BuyOffer {
    // ...
    uint64 buyFeeSnapshot; // fee locked at creation
```

```
}

// In createBuyOffer:
_offer.buyFeeSnapshot = buyFee;

// In calculateAmountSellerGets: replace global buyFee with _offer.buyFeeSnapshot
fee = (amountSellerGets * offer.buyFeeSnapshot) / 1e10;
```

Status

Resolved

[M-03] Authority Manager Circular Dependency — Silent Protocol Takeover

Description

The `onlyOwner` modifier resolves the owner dynamically via `authorityManager.owner()`. The `setAuthorityManager()` function allows the current owner to replace `authorityManager` with any contract. An owner who calls `setAuthorityManager(maliciousContract)` — where `maliciousContract.owner()` returns an attacker address — permanently transfers all admin control to the attacker. No event is emitted, making the change invisible to P2P contract monitoring.

Vulnerability Details

```
// P2P.sol:57-60
modifier onlyOwner() {
    require(msg.sender == authorityManager.owner(), "Not owner");
    _;
}

// P2P.sol:106-109
function setAuthorityManager(address _authorityManager) external onlyOwner {
    require(_authorityManager != address(0), "invalid address");
    authorityManager = IAuthorityManager(_authorityManager);
    // ← NO EVENT
}
```

After a malicious authority swap:

- `setBuyFee`, `setSellFee`, `setPriceMarkup` → controlled by attacker
- `setGoldPriceAggregator`, `setSilverPriceAggregator` → attacker manipulates all pricing
- All future `authorityManager.owner()` fee recipients → attacker's wallet

The circular dependency means the authority defines who the owner is, and the owner can change the authority — one call achieves full takeover.

Proof of Concept

```
// Attacker deploys:
contract MaliciousAuthority {
    function owner() external view returns (address) { return attacker; }
    // Implement other IAuthorityManager functions minimally
}

// Compromised owner calls:
p2p.setAuthorityManager(address(maliciousAuthority));
// → authorityManager.owner() now returns attacker
// → All onlyOwner functions now controlled by attacker
// → No P2P events emitted
```

Impact

Single-call, undetectable transfer of all admin privileges. Combined with M-02 and L-03, attacker can drain all trade value. All off-chain monitoring of P2P contract events will miss the takeover.

Recommendation

1. Emit an event on setAuthorityManager changes.
2. Implement a two-step transfer with a timelock delay.
3. Cache authorityManager in a local variable at function start for all functions making multiple calls.

Status

Acknowledged

[M-04] ERC3643 Buyer Compliance Loss Permanently Blocks Order Fill

Description

`acceptBuyOffer` transfers property tokens directly to `offer.maker` via `safeTransferFrom`. ERC3643 tokens enforce compliance checks on every transfer. If the buyer (`offer.maker`) loses whitelist status after creating the order, all fill attempts by any seller revert. The order is stuck and sellers waste gas indefinitely.

Vulnerability Details

```
// P2P.sol:433
token.safeTransferFrom(msg.sender, offer.maker, _tokenAmount);
// ↑ ERC3643 enforces compliance on the RECIPIENT (offer.maker)
// If offer.maker is not compliant, this reverts for ALL sellers
```

There is no pre-fill compliance check, no expiration mechanism, and no notification to sellers that the order is unfillable. `cancelBuyOffer` is available to the buyer but requires the buyer to be aware of the compliance loss and have key access.

Proof of Concept

1. Alice (ERC3643 compliant) creates a buy order. Order enters PENDING status.
2. Alice's compliance token is revoked (e.g., failed KYC renewal, jurisdiction change).
3. Bob calls `acceptSellOffer` for Alice's order.
4. `token.safeTransferFrom(Bob, Alice, amount)` reverts — Alice is non-compliant.
5. Bob loses gas. Order remains PENDING. No seller can fill it.
6. If Alice loses private key access, funds are locked until contract upgrade.

Impact

Gas waste for all sellers attempting to fill. Buyer funds locked if buyer loses key access. At scale, a compliance event affecting multiple buyers could lock significant protocol liquidity.

Recommendation

1. Add a compliance pre-check before executing the fill:

```
// Pseudo-code – depends on ERC3643 interface  
require(token.canTransfer(offer.maker, _tokenAmount), "Buyer not compliant");
```

2. Add offer expiration timestamps so stale/blocked orders self-expire.

Status

Acknowledged



[M-05] ERC3643 Property Price Decimal Field Ignored — Incorrect Payment Calculation

Description

`PropertyData` contains a `decimals` field indicating the decimal precision of `property.price`. This field is never read. The payment formula hardcodes a 10^6 divisor, assuming all prices are in D6 format. If the authority manager stores property prices in D8 format (common for Chainlink-derived prices), buyers are overcharged by 100x, and sellers receive 100x less than expected.

Vulnerability Details

```
// P2P.sol:202-203 (createSellOffer for ERC3643)
property = authorityManager.propertyDetails(_propertyId);
tokenPrice = property.price; // ← raw price, decimals ignored

// P2P.sol:327-329 (calculateForBuyOffer for ERC3643)
property = authorityManager.propertyDetails(_propertyId);
tokenPrice = property.price; // ← same, decimals not normalized

// P2P.sol:346-348 - hardcoded 10^6 divisor
buyAmount = (_tokenAmount * _price * (10 ** buyTokenDecimals)) /
             (10 ** tokenDecimals * 10 ** 6); // ← assumes price is D6
```

If `property.price` is in D8 (\$3,000.00000000 = 300000000000):

```
Expected buyAmount for 1 token: $3,000 USDC = 3,000,000 units (6 decimals)
Actual buyAmount: (1 * 300000000000 * 1e6) / (1e6 * 1e6) = 300,000,000,000,000 / 1e12
= 300,000
```

Buyer would pay \$300,000 USDC for a \$3,000 token — 100x overcharge.

Proof of Concept

1. Authority manager stores `property.price` = 300000000000 (D8, = \$3,000).
2. Seller creates an ERC3643 sell order. `_validatePrice` compares user price against D8 oracle → all prices appear `< tokenPrice - markup` → revert (can't create any offer at realistic prices).

3. Alternatively, if authority stores `property.price = 3000000` (D6) but decimals field says D8, the `decimals` field is meaningless and operators are confused.

Impact

All ERC3643 order creation or fill may fail or produce drastically wrong payment amounts depending on how the authority manager formats prices. The decimal normalization step is completely absent.

Recommendation

Normalize `property.price` using `property.decimals` before use:

```
uint256 tokenPrice;  
if (property.decimals > 6) {  
    tokenPrice = property.price / (10 ** (property.decimals - 6));  
} else if (property.decimals < 6) {  
    tokenPrice = property.price * (10 ** (6 - property.decimals));  
} else {  
    tokenPrice = property.price;  
}
```

Status

Acknowledged

[M-06] Owner Can Replace Price Oracle With Controlled Contract

Description

`setGoldPriceAggregator()` and `setSilverPriceAggregator()` allow the owner to replace the Chainlink oracle with any contract implementing `IPriceAggregator`. There is no timelock, no validation that the new oracle is a legitimate Chainlink feed, and no event emitted. A compromised or malicious owner can install a fake oracle returning arbitrary prices, enabling manipulation of all offer creation and price validation.

Vulnerability Details

```
// P2P.sol:116-128
function setGoldPriceAggregator(address _goldPriceAggregator) external onlyOwner {
    require(_goldPriceAggregator != address(0), "invalid address");
    goldPriceAggregator = IPriceAggregator(_goldPriceAggregator);
    // ← NO EVENT, NO TIMELOCK, NO VALIDATION
}
```

With a malicious oracle returning `price = 10000 * 1e8` (\$10,000 for gold):

- `_validatePrice` allows sellers to list up to \$11,000/token (oracle + 10% markup).
- Buyers calling `acceptSellOffer` pay \$11,000 + 10% fee = \$12,100 — believing gold is at \$10,000.
- Real gold at \$3,000 → buyers overpay 4x.

Proof of Concept

```
contract FakeOracle {
    function decimals() external pure returns (uint8) { return 8; }
    function latestRoundData() external pure returns (
        uint80, int256, uint256, uint256, uint80
    ) {
        return (1, 10000e8, 0, block.timestamp, 1); // reports $10,000 gold
    }
}

// Owner calls:
p2p.setGoldPriceAggregator(address(fakeOracle));
```

```
// All gold orders now priced at $10,000
```

Impact

Owner can manipulate all token pricing. Sellers can be forced to accept below-market prices; buyers can be lured into above-market purchases. The attack is invisible (no event emitted) until someone checks the oracle address on-chain.

Recommendation

1. Emit an event on oracle changes: `emit GoldOracleUpdated(oldOracle, newOracle).`
2. Add a timelock delay (e.g., 48 hours) before oracle changes take effect.
3. Validate the new oracle returns a sane price within bounds of the current oracle.

Status

Resolved

Low

[L-01] Residual Buyer Funds Sent to Owner Instead of Buyer

Description

When a buy order is completely filled and a `remainingPaymentAmount > 0` remains (due to integer rounding or fee changes), the leftover is transferred to `authorityManager.owner()` instead of being refunded to `offer.maker`. The buyer deposited these funds and is entitled to any unspent remainder.

Vulnerability Details

```
// P2P.sol:435-442
if (offer.filledTokens == offer.totalTokens) {
    if (offer.remainingPaymentAmount > 0) {
        offer.status = P2PLibrary.OrderStatus.FILLED;
        buyToken.safeTransfer(
            authorityManager.owner(), // ← should be offer.maker
            offer.remainingPaymentAmount
        );
    }
}
```

Under normal operation the residual is a few wei of rounding dust. When combined with M-02 (fee lowered after order creation), the residual equals the entire pre-funded fee deposit — potentially thousands of dollars.

Proof of Concept

1. Alice creates a buy order pre-funding \$11,000 USDC (`buyFee = 10%`).
2. Owner calls `setBuyFee(0)`.
3. Sellers completely fill the order. `_totalDeduction = $10,000`.
4. `remainingPaymentAmount = $1,000`. `filledTokens == totalTokens`.
5. L439: `safeTransfer(owner, $1,000)` — Alice's \$1,000 fee deposit goes to the owner.

Impact

Low standalone (rounding dust, typically a few wei). Amplified to high-value extraction when combined with M-02.

Recommendation

```
buyToken.safeTransfer(offer.maker, offer.remainingPaymentAmount);
```

Status

Acknowledged

[L-02] Sell Order Maker Receives Less Than Listed Price With No On-Chain Disclosure

Description

When a seller creates a sell order at price P , they expect to receive $\text{totalTokens} * P$ USDC. At fill time, a `sellFee` is deducted from their payout with no prior disclosure. `createSellOffer` neither calculates nor emits the net payout. The seller's tokens are locked in escrow at creation time — before they can discover the actual payout.

Vulnerability Details

`createSellOffer` stores `_price` but never calls `calculateForSellOffer` (which exists at L241 as dead code and computes the net payout):

```
// P2P.sol:212-213
_offer.price = _price;
_offer.totalTokens = _tokenAmount;
// calculateForSellOffer is never called here - seller receives no net payout
information
```

At fill time (`acceptSellOffer`):

```
// P2P.sol:511-514
uint256 _buyerFee = _fee;
uint256 _sellerFee = _fee;
uint256 _totalFee = _buyerFee + _sellerFee; // 2 * fee
uint256 _sellerReceive = _totalPaymentAmount - _totalFee; // seller gets base - 1 fee
```

****Concrete example (10 GOLD at \$2,800, `sellFee` = 10%):****

Seller lists at \$2,800/token expecting:	\$28,000
Buyer pays (base + 1 fee):	\$30,800
Protocol takes (2× fee):	\$ 5,600
Seller receives (base - 1 fee):	\$25,200 ← \$2,800 less than expected

Proof of Concept

1. Alice calls `createSellOffer` listing 10 GOLD at \$2,800 each, expecting to receive \$28,000 USDC.

2. Her tokens are locked in escrow at creation time with no net payout displayed or emitted.
3. Bob calls `acceptSellOffer(sellOfferId, 10e6)`.
4. `calculateAmountBuyerPays` returns `_totalPaymentAmount = $30,800, _fee = $2,800`.
5. `_sellerReceive = $30,800 - $5,600 = $25,200` — Alice receives \$2,800 less than her listed price.
6. Alice had no on-chain indication of this deduction before locking her tokens at step 1.

Impact

Every seller who does not independently call `calculateForSellOffer` off-chain loses `totalTokens * price * sellFee / 1e10` per filled order, guaranteed.

Recommendation

1. Call `calculateForSellOffer` inside `createSellOffer` and emit the net payout in `SellOfferCreated`.
2. Store `expectedPayout` in `SellOffer` struct so sellers and frontends can display it.
3. Or: define `_price` as the NET price the seller receives, and gross up for the buyer.

Status

Acknowledged

[L-03] Owner Can Set Protocol Fee Up to 100% With No Cap or Timelock

Description

All three fee setters cap their parameters at $1e10$, equivalent to 100% when divided by the protocol's $1e10$ denominator. The owner can raise fees tenfold (from default 10% to 100%) with a single transaction and zero notice to active order makers.

Vulnerability Details

```
// P2P.sol:91-103
function setBuyFee(uint64 _buyFee) external onlyOwner {
    require(_buyFee <= 1e10, "Fee percentage too high"); // 100% max
    buyFee = _buyFee;
}
```

At $\text{sellFee} = 1e10$ (100%), $\text{_totalFee} = 2 \times \text{fee} = 2 \times \text{buyAmount}$. $\text{_sellerReceive} = 2 \times \text{buyAmount} - 2 \times \text{buyAmount} = 0$. Seller receives nothing; owner takes the entire buyer payment.

With $\text{priceMarkup} = 1e10$ (100% above oracle): $\text{max listing} = \text{oracle} \times 2$. Buyer pays $\text{oracle} \times 2 \times 2 = 4 \times \text{oracle}$ at extreme conditions.

Proof of Concept

1. Owner calls `setSellFee(1e10)`.
2. Bob's existing sell order for 10 GOLD at \$2,800 is still open.
3. Alice calls `acceptSellOffer` unaware of the fee change.
4. Alice pays \$56,000. Bob receives \$0. Owner receives \$56,000.

Impact

Low (centralization risk — requires malicious/compromised owner). Combined with M-02 and M-03, this is a complete protocol rug vector requiring only the owner key.

Recommendation

1. Set a reasonable fee cap: `require(_buyFee <= 1e9, "Fee exceeds 10%")`.
2. Add a timelock on fee changes so users can cancel open orders before the new fee activates.

Status

Resolved

[L-04] Arbitrary ERC20 Token Address in Sell Orders — Token Impersonation and Order Book DoS

Description

For ERC20-type sell orders, `createSellOffer` stores the user-supplied `_tokenAddress` directly without verifying it is a factory-registered asset. Buy orders, by contrast, source their token address from `factory.getTokenInfo`, not user input.

Vulnerability Details

```
// P2P.sol:220-222
_offer.tokenAddress = _propertyType == P2PLibrary.PropertyType.ERC3643
    ? property.token
    : _tokenAddress; // ← user-supplied, not factory-verified
```

The `factory.getTokenInfo(_tokenAddress)` call at L205 only determines which oracle to use; it does not gate acceptance of the address.

****Attack vectors:****

- **Token impersonation:**** Attacker creates sell order with `_tokenAddress = USDC`, priced at Gold oracle (\$3,000/unit). Naive buyer accepts, pays \$3,000,000 USDC, receives \$1,000 USDC. Attacker extracts the difference.
- **Order book DoS:**** Attacker floods `sellOffers` mapping with junk addresses (1 wei each). `sellOfferCounter` only ever increments. No purge mechanism. Indexers/frontends enumerating all sell orders are permanently burdened.

Proof of Concept

```
createSellOffer(
    BuyType.USDC,
    PropertyType.ERC20,
    0,
    1000e6, // 1000 "tokens" (actually USDC units)
```

```
3000_000000, // $3,000 per token – passes _validatePrice via Gold oracle
USDC_ADDRESS // arbitrary token, not registered Gold
);
// Naive buyer accepts, paying 1000 * $3,000 * 1.1 = $3,300,000 USDC
// Buyer receives 1000 USDC ($1,000)
```

Impact

Low-to-Medium depending on frontend validation. Naive frontends that display only price and amount without validating the underlying token contract expose users to impersonation. Order book DoS has no recovery mechanism once started.

Recommendation

```
P2PLibrary.TokenInfo memory _tokenInfo = factory.getTokenInfo(_tokenAddress);
require(_tokenInfo.tokenAddress != address(0), "Token not registered");
_offer.tokenAddress = _tokenInfo.tokenAddress;
```

Status

Resolved

[L-05] Proxy `receive()` Traps ETH Permanently — No Withdrawal Path

Description

`UpgradableProxy` defines a `receive()` function that accepts plain ETH transfers without delegating to the implementation. No ETH withdrawal function exists anywhere in the proxy. Any ETH sent to the proxy address is permanently locked.

Vulnerability Details

```
// proxy/ProxyContract.sol:78-80
receive() external payable {
    // ETH received successfully
}
```

The base Proxy contract's `_fallback()` would normally delegate ETH calls to the implementation. By overriding `receive()`, the proxy silently accepts ETH and does nothing. The implementation (`P2P.sol`) has no payable functions and no ETH recovery mechanism.

Proof of Concept

```
// User accidentally sends 1 ETH to the proxy address
(bool sent,) = address(proxy).call{value: 1 ether}("");
// sent = true - receive() accepts the ETH silently
// No withdrawal function exists anywhere in the proxy or implementation
// 1 ETH is permanently locked
```

Impact

Low — affects only accidental ETH sends to the proxy address. No exploit path. Common enough user error (sending ETH to a USDC/USDT protocol address) to warrant a fix.

Recommendation

Remove the `receive()` override to let the base Proxy delegate ETH calls, or add an explicit ETH refund:

```
receive() external payable {  
    revert("ETH not accepted");  
}
```

Status

Resolved



[L-06] Non-Upgradeable ReentrancyGuard Behind Upgradeable Proxy

Description

P2P.sol imports the non-upgradeable ReentrancyGuard from OpenZeppelin (@openzeppelin/contracts/utils/ReentrancyGuard.sol). The _status storage variable is initialized to _NOT_ENTERED (1) in its constructor. When deployed behind a proxy, the constructor does not run — _status starts at 0 (uninitialized default). The reentrancy check (require(_status != _ENTERED) = require(_status != 2)) passes with _status = 0, so the first call is functionally safe but architecturally incorrect.

Vulnerability Details

- Expected initial state: _status = 1 (_NOT_ENTERED)
- Actual initial state behind proxy: _status = 0
- First nonReentrant call: require(0 != 2) passes, sets _status = 2, then resets to 1
- Subsequent calls behave correctly

No current security impact. However, using non-upgradeable patterns in upgradeable contracts is an architectural defect that can cause subtle bugs during future upgrades if storage layout is modified.

Proof of Concept

```
// Behind proxy: ReentrancyGuard constructor never runs
// _status storage slot starts at 0 (default) instead of 1 (_NOT_ENTERED)

// First nonReentrant call:
// require(_status != 2) → require(0 != 2) → PASSES (correct result, wrong reason)
// _status set to 2, function executes, _status reset to 1

// Subsequent calls work correctly (1 != 2)
// But storage slot 0 was incorrectly used as "uninitialized" sentinel on first call
```

Impact

Low — no functional reentrancy bypass under current code. The check _status != 2 passes correctly with either 0 or 1. Risk materializes if a future upgrade adds logic that

explicitly checks for `_status == 1` as a precondition, or if the storage layout is reorganized such that `_status` slot overlaps with another variable.

Recommendation

Replace with the upgradeable variant and initialize properly:

```
import "@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";

contract P2P is Initializable, ReentrancyGuardUpgradeable {
    function initialize(...) external initializer {
        __ReentrancyGuard_init();
        ...
    }
}
```

Status

Resolved

[L-07] No Events Emitted on Any Admin Parameter Changes

Description

Seven admin setter functions in `P2P.sol` modify critical protocol parameters (fees, oracles, authority, factory) without emitting any events. Off-chain monitoring tools, frontends, and users have no on-chain mechanism to detect parameter changes.

Vulnerability Details

```
// P2P.sol:91-129 - all 7 functions emit no events:
setBuyFee()           // changes fee charged on all buy orders
setSellFee()          // changes fee charged on all sell orders
setPriceMarkup()      // changes allowed price deviation from oracle
setAuthorityManager() // changes who the "owner" is (see M-03)
setFactory()          // changes token registry
setGoldPriceAggregator() // changes gold price source (see M-06)
setSilverPriceAggregator() // changes silver price source
```

Proof of Concept

```
// Owner silently raises buyFee from 10% to 50%
p2p.setBuyFee(5e9);
// No event emitted - Etherscan event log shows nothing
// Monitoring tools, indexers, and users are completely unaware
// Next acceptBuyOffer call on any existing order → underflow revert (M-02)
// Users have no warning until their transaction fails
```

Impact

Low — no direct exploit path. Users cannot monitor parameter changes without polling storage. A fee change or oracle swap (M-02, M-06) is invisible until a trade is affected.

Recommendation

Add events for all admin state changes:

```
event BuyFeeUpdated(uint64 oldFee, uint64 newFee);
event SellFeeUpdated(uint64 oldFee, uint64 newFee);
```

```
event AuthorityManagerUpdated(address indexed oldManager, address indexed  
newManager);  
// etc.
```

Status

Resolved



[L-08] Single-Step Proxy Ownership Transfer — No Pending/Confirm Pattern

Description

`UpgradableProxy.transferOwnership()` transfers proxy ownership immediately in one transaction. Sending to a wrong address (typo, clipboard hijack) results in permanent and irrecoverable loss of proxy control, including the ability to upgrade the implementation.

Vulnerability Details

```
// proxy/ProxyContract.sol:182-187
function transferOwnership(address newOwner) public virtual onlyProxyOwner {
    if (newOwner == address(0)) revert ProxyOwnableInvalidOwner(address(0));
    _transferOwnership(newOwner); // ← immediate, no confirmation step
}
```

Proof of Concept

```
// Operator intends to transfer to 0xAlice but pastes wrong address
proxy.transferOwnership(0xDead000000000000000000000000000000000000);
// Ownership immediately transferred to dead address
// No confirmation step – cannot be reversed
// proxy.setImplementation() / upgradeToAndCall() permanently inaccessible
```

Impact

Low — requires operator error. Consequence is permanent lockout from all proxy admin functions.

Recommendation

Implement the two-step ownership pattern (pending + accept):

```
address public pendingOwner;

function transferOwnership(address newOwner) public onlyProxyOwner {
    pendingOwner = newOwner;
}
```

```
function acceptOwnership() public {  
    require(msg.sender == pendingOwner, "Not pending owner");  
    _transferOwnership(pendingOwner);  
    pendingOwner = address(0);  
}
```

Status

Acknowledged

[L-09] `setPrice()` Mock Function Exposed in Production Oracle Interface

Description

`IPriceAggregator.sol` includes a `setPrice(int256 newPrice)` function alongside the standard Chainlink view functions. This is a mock/test helper that was not removed from the production interface. Its presence signals to integrators and auditors that oracle prices can be set programmatically, which is both misleading and a potential attack surface if any deployed contract implements this function.

Vulnerability Details

```
// interfaces/IPriceAggregator.sol:18
function setPrice(int256 newPrice) external;
```

Any address satisfying `IPriceAggregator` (as validated by `setGoldPriceAggregator`) can implement `setPrice`. A deployed oracle that accidentally exposes a callable `setPrice` would allow anyone to manipulate gold/silver prices in the protocol.

Proof of Concept

```
// A contract satisfying IPriceAggregator but with publicly callable setPrice:
contract ExposedOracle {
    int256 public price = 3000e8;
    function decimals() external pure returns (uint8) { return 8; }
    function latestRoundData() external view returns (
        uint80, int256, uint256, uint256, uint80
    ) { return (1, price, 0, block.timestamp, 1); }
    function setPrice(int256 p) external { price = p; } // unguarded - anyone can
    call
}
// Owner installs it: p2p.setGoldPriceAggregator(address(exposedOracle));
// Any attacker calls: exposedOracle.setPrice(10000e8); // $10,000 gold
// All new offers validated against $10,000 - buyers pay 3x market rate
```

Impact

Low — no immediate exploit with Chainlink feeds. Risk if a non-standard oracle is deployed.

Recommendation

Remove `setPrice` from the production interface. Maintain a separate mock interface for testing:

```
// interfaces/IPriceAggregator.sol - production
interface IPriceAggregator {
    function decimals() external view returns (uint8);
    function latestRoundData() external view returns (...);
}

// mocks/IMockPriceAggregator.sol - test only
interface IMockPriceAggregator is IPriceAggregator {
    function setPrice(int256 newPrice) external;
}
```

Status

Resolved

[L-10] No Offer Expiration Mechanism — Stale Orders Persist Indefinitely

Description

Neither `BuyOffer` nor `SellOffer` contains an expiration timestamp. Offers created at a given oracle price remain active indefinitely. As gold/silver prices drift, stale offers at outdated prices can be exploited: buyers accept sell offers priced below current market, or sellers fill buy orders priced above current market.

Vulnerability Details

`_validatePrice` is only called at offer creation, not at fill time. Once an offer is PENDING, its stored `price` is final regardless of market movement:

```
// P2P.sol:303-311
function _validatePrice(uint256 _price, uint256 _tokenPrice) internal view {
    // Only called in createBuyOffer and createSellOffer
    // NOT called in acceptBuyOffer or acceptSellOffer
}
```

Proof of Concept

1. Alice creates a sell order for GOLD at \$3,300 (oracle \$3,000 + 10% markup = maximum allowed).
2. Gold market price falls to \$2,700 over the next month.
3. The sell order remains PENDING with `offer.price` = \$3,300 — no expiry enforced.
4. Bob calls `acceptSellOffer`, buying at \$3,300 + 10% fee = \$3,630/token.
5. Bob overpays by \$930/token relative to current market (\$2,700).

Impact

Low — requires passive market movement. Orders created during price spikes remain exploitable after prices normalize. The longer the horizon, the greater the potential mispricing.

Recommendation

Add `expiry` to offer structs and enforce at fill time:

```
struct BuyOffer {  
    // ...  
    uint256 expiry; // unix timestamp  
}  
  
// In acceptBuyOffer:  
require(block.timestamp <= offer.expiry || offer.expiry == 0, "Offer expired");
```

Status

Acknowledged

[L-11] No Minimum Fill Amount — Enables Gas Griefing and Zero-Payment Fills

Description

`acceptBuyOffer` and `acceptSellOffer` only require `_tokenAmount > 0`. Any positive amount — including 1 wei — is accepted. This enables two attack vectors: (1) spam fills that waste seller/buyer gas and fragment order accounting, and (2) combined with H-01 (zero-payment truncation for 18-decimal tokens), repeated 1-unit fills drain a sell offer for free.

Vulnerability Details

```
// P2P.sol:393, 492 (acceptBuyOffer, calculateAmountBuyerPays)
require(_tokenAmount > 0, "Invalid token amount");
// ← No minimum fill, no anti-spam protection
```

For 6-decimal gold tokens, a `_tokenAmount = 1` fill pays \$0.003 USDC (non-zero but extremely small). For 18-decimal tokens, it pays \$0 (see H-01). In both cases, many micro-fills drain gas from counterparties.

Proof of Concept

```
// Attacker targets a sell order with 18-decimal tokens (H-01 precondition):
uint256 totalTokens = sellOffers[offerId].totalTokens;
for (uint256 i = 0; i < totalTokens; i++) {
    p2p.acceptSellOffer(offerId, 1); // buyAmount = 0 → pays $0 USDC, receives 1
    token unit
}
// Cost: gas only (~$0.01/tx on L2 × totalTokens)
// Profit: entire token supply of the sell order

// For 6-decimal tokens (not zero-payment but spam-viable):
for (uint256 i = 0; i < 10000; i++) {
    p2p.acceptSellOffer(offerId, 1); // pays $0.003 USDC, forces seller into 10,000
    partial fills
}
```

Impact

Low — gas griefing and amplification vector for H-01.

Recommendation

Add a minimum fill amount at offer creation time:

```
struct BuyOffer {  
    // ...  
    uint256 minimumFillAmount;  
}  
  
// In acceptBuyOffer:  
require(_tokenAmount >= offer.minimumFillAmount, "Below minimum fill");
```

Status

Acknowledged

QA

[Q-01] calculateForSellOffer Is Dead Code — Never Called Internally

Description

calculateForSellOffer (L241-259) computes the seller's net payout after fee deduction. It is declared public view but is never called internally by createSellOffer, acceptSellOffer, or any other function. It was clearly built to disclose the seller's net payout at order creation time but was never integrated. Its absence directly contributes to L-02.

Vulnerability Details

```
// P2P.sol:241-259
function calculateForSellOffer(
    P2PLibrary.BuyType _buyType,
    uint256 _tokenAmount,
    uint256 _price,
    address _tokenAddress
) public view returns (uint256 _amount, uint256 fee) {
    // Computes: _amount = gross payout; fee = sellFee deduction; _amount -= fee
    // Never invoked by createSellOffer, acceptSellOffer, or any other internal
function
}
```

Searching all call sites: only the function definition at L241 exists. Zero internal callers.

Proof of Concept

```
grep -n "calculateForSellOffer" P2P.sol
# Output:
# 241: function calculateForSellOffer(...) public view returns (...) {
# - No other lines found. Zero callers.
```

Impact

No direct security impact. Frontend developers relying on this function for payout estimates receive a value computed with the current global `sellFee`, which may differ from the fee at actual fill time (see M-02), giving incorrect estimates.

Recommendation

Integrate into `createSellOffer` to compute and emit the seller's net expected payout, or remove to avoid misleading future readers.

Status

Acknowledged

[Q-02] `factory.getTokenInfo` Returns Empty Struct for Unregistered Tokens Without Reverting

Description

In `createSellOffer` (L205) and `calculateForBuyOffer` (L331), `factory.getTokenInfo(_tokenAddress)` is called with user-supplied input. For unregistered tokens, the function returns an empty `TokenInfo` struct with `assetType = 0` (`AssetType.Gold`), causing the Gold oracle to price any unregistered token. No `require(info.tokenAddress != address(0))` guard exists.

Vulnerability Details

```
// P2P.sol:205-206
_tokenInfo = factory.getTokenInfo(_tokenAddress); // returns empty struct if
unregistered
tokenPrice = getAssetPrice(_tokenInfo.assetType); // assetType = 0 (Gold) for
unknown tokens

// P2P.sol:331-334
P2PLibrary.TokenInfo memory info = factory.getTokenInfo(_tokenAddress);
tokenPrice = getAssetPrice(info.assetType); // same issue - Gold oracle used silently
tokenAddress = info.tokenAddress; // returns address(0) for unregistered
tokens
```

No validation that `info.tokenAddress != address(0)` exists at either call site.

Proof of Concept

```
// createSellOffer with unregistered token address:
createSellOffer(BuyType.USDC, PropertyType.ERC20, 0, 100e6, 3000_000000,
randomToken);
// factory.getTokenInfo(randomToken) returns TokenInfo{tokenAddress: address(0),
assetType: 0}
// assetType = 0 = AssetType.Gold → Gold oracle prices the order
// Order created successfully with arbitrary token priced as Gold
```

Impact

Silent mispricing of unregistered tokens against the Gold oracle. Enables the impersonation attack in L-04 by making price validation pass for any token. Buyers may accept orders for worthless tokens priced at spot gold rates.

Recommendation

```
P2PLibrary.TokenInfo memory info = factory.getTokenInfo(_tokenAddress);  
require(info.tokenAddress != address(0), "Token not registered in factory");
```

Status

Resolved

[Q-03] Fee-on-Transfer Tokens Not Accounted For — Balance-Delta Pattern Missing

Description

`createSellOffer` (L232) and `createBuyOffer` (L178) record `totalTokens/paymentAmount` before `safeTransferFrom` executes. For fee-on-transfer tokens, the contract receives less than the recorded value. Subsequent operations (`cancelSellOffer`, `acceptBuyOffer`) use the recorded value and would attempt to transfer more than the contract actually holds.

The developer acknowledged this at L186: `"Not considering Fee-On Transfers heavily here given the tokens will be GOLD/SILVER or USDC/USDT."` USDT contains a configurable transfer fee mechanism (currently 0%), so this is a latent risk.

Vulnerability Details

```
// P2P.sol:214 - amount recorded BEFORE transfer
_offer.totalTokens = _tokenAmount; // recorded: 1,000

// P2P.sol:232-236 - actual transfer may receive less
IERC20Metadata(_offer.tokenAddress).safeTransferFrom(
    msg.sender, address(this), _tokenAmount // sent: 1,000, received: 990 (if 1% fee)
);
// offer.totalTokens = 1,000 but contract only holds 990
```

Proof of Concept

Scenario: USDT enables its transfer fee at 1%.

1. Seller calls `createSellOffer` with `totalTokens = 1,000` USDT.
2. Contract records `offer.totalTokens = 1,000`.
3. Contract receives 990 USDT (10 taken as fee).
4. `cancelSellOffer`: `safeTransfer(maker, 1,000)` → contract only has 990 → REVERT.
5. Seller's 990 USDT is permanently locked in the contract.

Impact

Latent risk with current in-scope tokens (USDT transfer fee is currently 0% on mainnet). If USDT enables its fee, all active sell orders become uncancellable. Partial fills may drain other offers' balances due to the shared payment pool.

Recommendation

Use balance-before/after pattern to record actual received amounts:

```
uint256 before = token.balanceOf(address(this));  
token.safeTransferFrom(msg.sender, address(this), _tokenAmount);  
_offer.totalTokens = token.balanceOf(address(this)) - before;
```

Status

Acknowledged

[Q-04] Fee Calculation Rounds Down — Systematic Dust Accumulation in Buy Order Escrow

Description

All three fee calculations use integer division, truncating toward zero:

```
fee = (buyAmount * buyFee) / 1e10 // rounds down
```

Across multiple partial fills, each fill independently rounds down its `_totalDeduction`. The sum of deductions is slightly less than the initial `remainingPaymentAmount`, leaving a non-zero dust remainder after full fill. This dust is the mechanical cause of L-01 (wrong recipient). Rounding always favors the user over the protocol — no security impact in isolation.

Vulnerability Details

```
// P2P.sol:350 (createBuyOffer path)
fee = (buyAmount * buyFee) / 1e10; // initial pre-fund calculation - rounds down

// P2P.sol:483 (acceptBuyOffer path - per partial fill)
fee = (amountSellerGets * buyFee) / 1e10; // per-fill deduction - rounds down
independently

// Sum of N partial fill deductions ≤ single full-fill calculation
// Difference accumulates as dust in remainingPaymentAmount
```

Proof of Concept

```
totalTokens = 3, price = $1 each, buyFee = 33.33% (3_333_333_333):
Initial paymentAmount = $3 + floor(3 * 0.3333) = $3 + $0 = $3 (fee rounds to 0 in wei
units)

Fill 1 (1 token): amountSellerGets = $1, fee = floor($1 * 0.3333) = $0
Fill 2 (1 token): amountSellerGets = $1, fee = $0
Fill 3 (1 token): amountSellerGets = $1, fee = $0
```

Total deducted = \$3. remainingPaymentAmount = \$0 – no dust **in** this **case**.

With non-trivial amounts the rounding produces 1–5 wei dust per fill cycle, triggering the L-01 wrong-recipient transfer on full fill.

Impact

Negligible in isolation — dust amounts, typically 1–5 wei per fully-filled order. Rounding always favors the user over the protocol. The primary consequence is triggering L-01 (dust sent to wrong address).

Recommendation

No isolated code change needed. Fix L-01's wrong recipient (offer.maker instead of owner) to ensure any rounding dust is correctly refunded to the buyer.

Status

Acknowledged

[Q-05] oracleDecimals < 6 Branch in getAssetPrice Is Unreachable With Any Live Feed

Description

All Chainlink price feeds (including XAU/USD and XAG/USD) return 8 decimal places. The `oracleDecimals < 6` branch is dead code. No security impact — the branch would execute correctly if ever reached.

Vulnerability Details

```
// P2P.sol:372 - oracle decimals check
uint8 oracleDecimals = priceAggregator.decimals(); // Chainlink XAU/USD: 8, XAG/USD:
8

if (oracleDecimals > 6) {                                // 8 > 6 → TRUE - always executes
    normalizedPrice = normalizedPrice / (10 ** (oracleDecimals - 6)); // divides by
100
} else if (oracleDecimals < 6) {                          // 8 < 6 → FALSE - never executes
    normalizedPrice = normalizedPrice * (10 ** (6 - oracleDecimals));
}
// If oracleDecimals == 6: both branches skipped (neither executes)
```

Proof of Concept

Chainlink XAU/USD `decimals()` = 8 → 8 > 6 → first branch always taken.
 Chainlink XAG/USD `decimals()` = 8 → same result.
 No live Chainlink USD price feed uses fewer than 6 decimals.
 The `else-if` branch can only execute with a custom non-standard oracle.

Impact

No security impact. The branch is syntactically correct and would execute properly if a sub-6-decimal oracle were ever configured. It is purely dead code under all current oracle configurations.

Recommendation

Remove the unreachable branch to reduce code complexity, or add a `revert` to make the assumption explicit:

```
if (oracleDecimals > 6) {  
    normalizedPrice = normalizedPrice / (10 ** (oracleDecimals - 6));  
} else if (oracleDecimals < 6) {  
    revert("Oracle decimals < 6 not supported");  
}
```

Status

Acknowledged

[Q-06] No Storage Gap Variables for Future Upgrades

Description

P2P.sol uses no `__gap` storage variables and does not implement ERC-7201 namespaced storage. Any future upgrade that adds or reorders inherited contracts will shift storage slot assignments, potentially corrupting critical state variables (`buyFee`, `sellFee`, `authorityManager`, offer mappings).

Vulnerability Details

```
// P2P.sol - Current storage layout (no gap):
// slot 0: _initialized (Initializable)
// slot 1: _status (ReentrancyGuard)
// slot 2: buyOfferCounter
// slot 3: sellOfferCounter
// slot 4: buyFee (packed with sellFee, priceMarkup)
// ...
// ← No uint256[50] private __gap; at end of contract
```

If a future upgrade inserts a new base contract between `Initializable` and `ReentrancyGuard`, all downstream slots shift, corrupting `buyFee`, `sellFee`, `authorityManager`, and all offer data.

Proof of Concept

```
Current: slot 1 = _status (ReentrancyGuard)
Upgrade adds "OwnableUpgradeable" before ReentrancyGuard:
New:     slot 1 = _owner (OwnableUpgradeable)
         slot 2 = _status (ReentrancyGuard, shifted)
         slot 3 = buyOfferCounter (was slot 2 - now misaligned)
→ buyOfferCounter reads stale _status value → contract behaves incorrectly
```

Impact

No current impact. Risk materializes only if a future upgrade modifies the inheritance chain without careful storage layout management. Without `__gap`, a single inheritance change silently corrupts all protocol state.

Recommendation

Add gap variables at the end of the contract's storage declarations:

```
uint256[50] private __gap;
```

Or adopt ERC-7201 namespaced storage for all state variables.

Status

Acknowledged

[Q-07] TOCTOU on Dual `authorityManager.owner()` Calls in `acceptBuyOffer`

Description

`acceptBuyOffer` calls `authorityManager.owner()` at two separate points (L427 for fee transfer, L439 for residual transfer) with three external token transfers between them. If `authorityManager` is changed between these calls (e.g., via a malicious token's `receive()` hook triggering `setAuthorityManager`), the fee and residual may be routed to different addresses.

Under current implementation: `nonReentrant` blocks re-entry into P2P, but `setAuthorityManager` does not re-enter P2P — it updates P2P's own storage. This is contrived but architecturally imprecise.

Vulnerability Details

```
// P2P.sol:427
buyToken.safeTransfer(authorityManager.owner(), _totalFee);      // owner() call #1

// P2P.sol:430 - safeTransfer to seller (external call)
buyToken.safeTransfer(msg.sender, _sellerAmount);

// P2P.sol:433 - safeTransferFrom property token (external call)
token.safeTransferFrom(msg.sender, offer.maker, _tokenAmount);

// P2P.sol:439
buyToken.safeTransfer(authorityManager.owner(), offer.remainingPaymentAmount); //
owner() call #2
// If authorityManager changed between L427 and L439, different address receives
residual
```

Proof of Concept

Precondition: `buyToken` is a malicious ERC20 with a `receive` hook (contrived – USDC/USDT don't have this).

1. `safeTransfer` at L427 triggers malicious `buyToken.transfer(ownerA, fee)`.
2. `buyToken.transfer` hook calls `p2p.setAuthorityManager(maliciousContract)`.

- (Not reentrancy `into` `p2p`; `s nonReentrant` path – just storage update)
3. `authorityManager.owner()` now returns `ownerB`.
 4. `safeTransfer` at L439: residual sent `to` `ownerB` instead `of` `ownerA`.

Impact

QA — extremely contrived. Requires the payment token (USDC/USDT) to execute arbitrary code during a transfer, which neither does. No realistic exploit path with in-scope tokens.

Recommendation

Cache `authorityManager.owner()` in a local variable at the start of the function:

```
address owner = authorityManager.owner();  
// use `owner` for all transfers
```

Status

Resolved

[Q-08] remainingPaymentAmount Retains Stale Non-Zero Value After Full Fill

Description

In `acceptBuyOffer` at L435-442, after transferring the residual to the owner, `offer.remainingPaymentAmount` is never set to 0. The offer's status is set to FILLED, but the storage field retains a non-zero value. Off-chain indexers reading `remainingPaymentAmount` for FILLED offers will see a misleading positive balance. Future upgrades that use `remainingPaymentAmount` as a guard may behave incorrectly.

Vulnerability Details

```
// P2P.sol:435-442
if (offer.filledTokens == offer.totalTokens) {
    if (offer.remainingPaymentAmount > 0) {
        offer.status = P2PLibrary.OrderStatus.FILLED;
        buyToken.safeTransfer(
            authorityManager.owner(),
            offer.remainingPaymentAmount
        );
        // ← offer.remainingPaymentAmount NOT zeroed after transfer
        // FILLED offer still shows non-zero remainingPaymentAmount in storage
    }
}
```

Proof of Concept

1. Alice's buy order: `totalTokens = 100`, `paymentAmount = $1,100` (base + 10% fee).
2. Order fully filled across 3 partial fills → 5 wei dust remains.
3. `acceptBuyOffer` final fill: `safeTransfer(owner, 5 wei)`, offer marked FILLED.
4. `offer.remainingPaymentAmount` is never zeroed – retains value 5.
5. Off-chain indexer reads offer struct:

<code>orderStatus</code>	<code>= FILLED</code>	← correct
<code>remainingPayment</code>	<code>= 5 wei</code>	← stale, should be 0
6. Indexer incorrectly reports "5 wei of payment still available" for a completed order.

Impact

No security impact. Misleads off-chain systems reading `remainingPaymentAmount` without also checking `status`. Future upgrades that rely on `remainingPaymentAmount == 0` as a completion guard (without checking `status`) could reprocess FILLED orders.

Recommendation

```
buyToken.safeTransfer(offer.maker, offer.remainingPaymentAmount);  
offer.remainingPaymentAmount = 0; // ← add this
```

Status

Resolved

[Q-09] Dual Ownership Model — Proxy Owner and P2P Authority Manager Are Fully Independent

Description

The proxy's `OWNER_SLOT` (controls `setImplementation`, `upgradeToAndCall`, `transferOwnership`) and P2P's effective owner (`authorityManager.owner()`, controls fees, oracles, factory) are completely independent. The two roles can be held by different entities with no documented relationship. Conflicting governance — e.g., proxy owner upgrading to an implementation that the P2P owner did not sanction — is possible.

Vulnerability Details

```
// proxy/ProxyContract.sol - Proxy authority
bytes32 internal constant OWNER_SLOT = keccak256("proxy.owner.slot");
modifier onlyProxyOwner() { ... } // controls: setImplementation, upgradeToAndCall,
transferOwnership

// P2P.sol - Protocol authority
modifier onlyOwner() {
    require(msg.sender == authorityManager.owner(), "Not owner");
    _;
} // controls: setBuyFee, setSellFee, setGoldPriceAggregator, setAuthorityManager,
etc.
// These are DIFFERENT addresses with no enforced relationship
```

Proof of Concept

Deployment: Team A holds `OWNER_SLOT` (proxy admin). Team B holds `authorityManager.owner()` (P2P admin).

Scenario - conflicting upgrade:

1. Team B calls `setSellFee(1e9)` expecting 10% fee on the current implementation.
2. Team A simultaneously calls `upgradeToAndCall(newImpl, "")` - new implementation changes fee semantics.
3. Team B's fee change lands on the new implementation with different behavior.

4. Neither team has visibility into the other's actions without off-chain coordination.

Impact

No direct security impact. Governance confusion between proxy owner and protocol owner can lead to misconfigured upgrades, split administrative authority, or uncoordinated parameter changes — all with no on-chain resolution mechanism.

Recommendation

Document the intended relationship between the two ownership roles clearly in natspec. Consider unifying under a single governance contract or requiring both owners to sign upgrades.

Status

Acknowledged

[Q-10] Silent Implementation Swap via `setImplementation()` — Missing ERC1967 Event

Description

`UpgradeableProxy.setImplementation()` swaps the implementation contract but emits no `Upgraded` event. The ERC-1967 standard and all standard monitoring tools (Etherscan, Tenderly, OpenZeppelin Defender) rely on the `Upgraded(address)` event to detect proxy upgrades. A proxy owner can silently redirect all contract calls — including escrow operations — to a malicious implementation, draining all USDC/USDT and property tokens without triggering any alert.

Vulnerability Details

```
// proxy/ProxyContract.sol:92-101
function setImplementation(address newImplementation) public onlyProxyOwner {
    if (newImplementation.code.length == 0) {
        revert ERC1967InvalidImplementation(newImplementation);
    }
    StorageSlot
        .getAddressSlot(IMPLEMENTATION_SLOT)
        .value = newImplementation;
    // ← NO EVENT EMITTED
}
```

The correct upgrade path `upgradeToAndCall()` DOES emit the event:

```
// proxy/ProxyContract.sol:134-146
function upgradeToAndCall(address newImplementation, bytes memory data) public
onlyProxyOwner {
    setImplementation(newImplementation);
    emit IERC1967.Upgraded(newImplementation); // ← event only here
    ...
}
```

Since `setImplementation` is public, a proxy owner can call it directly and bypass the event entirely.

Proof of Concept

```
// Attacker (compromised proxy owner) calls:
proxy.setImplementation(address(maliciousImpl));
// → No Upgraded event emitted
// → Monitoring tools see no upgrade
// → maliciousImpl.drainAll() extracts all USDC/USDT and property tokens
```

Impact

Undetectable rug pull. All escrowed funds (USDC, USDT, Gold/Silver tokens, ERC3643 property tokens) are at risk. Standard security monitoring that watches for Upgraded events will NOT detect the swap.

Recommendation

Remove the public upgrade path from setImplementation or make it emit the event:

```
function setImplementation(address newImplementation) internal {
    // make internal - only upgradeToAndCall can invoke it
    ...
}
```

Or add the event directly to setImplementation:

```
function setImplementation(address newImplementation) public onlyProxyOwner {
    if (newImplementation.code.length == 0) revert
    ERC1967InvalidImplementation(newImplementation);
    StorageSlot.getAddressSlot(IMPLEMENTATION_SLOT).value = newImplementation;
    emit IERC1967.Upgrade(newImplementation); // ← add this
}
```

Status

Resolved

Centralisation

The BlackTie Deal project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the BlackTie Deal project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. . Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.